

Below Zero: The Natural Language

The word “Home” is nothing but [0.75, -0.258, 0.25, 0.36...]

In this documentation, we will learn about the foundational mechanics of **Natural Language Processing** by building a **Word2Vec Skip-Gram** model entirely from scratch.

This guide is recommended for those who already understand the calculus behind backpropagation and gradient descent, and are highly comfortable managing pointers, structures, and dynamic memory allocations in pure C. By the end, you will see exactly how raw matrix multiplications and negative log-likelihood loss force semantic meaning into a geometric vector space.

What is NLP?

Natural language processing (NLP) is a subfield of computer science and [artificial intelligence \(AI\)](#) that uses [machine learning](#) to enable computers to understand and communicate with human language.

The Traditional Approach (The NLTK Era):

Before deep learning took over, NLP was dominated by statistical and rule-based libraries like NLTK (Natural Language Toolkit). This approach treated language like a rigid state machine. To process text, engineers relied on explicit, hardcoded pipelines:

- **Tokenization:** Splitting sentences by spaces and punctuation.
- **Stemming & Lemmatization:** Chopping off the ends of words to find the root (e.g., turning "running" into "run").
- **Stop-Word Removal:** Deleting common words like "the" or "and" to save memory.
- **Bag of Words & TF-IDF:** Counting how many times a specific word appeared in a document to determine its importance.

The traditional approach hit a hard mathematical ceiling because it was **semantically blind**. In a traditional system, words are represented via **One-Hot Encoding**. If you have a vocabulary of 50,000 words, every single word is represented as an array of 50,000 integers containing exactly 49,999 zeros and a single 1. This creates two fatal problems:

- **The Curse of Dimensionality:** Processing 50,000-dimensional sparse arrays destroys memory and computational efficiency.
- **Zero Geometric Meaning:** In a One-Hot encoded space, the dot product of the array for "King" and the array for "Queen" is exactly 0. To the traditional algorithm, "House" and "Home" are just as mathematically unrelated as "House" and "Helicopter". The machine could count words, but it had absolutely no idea what they meant.

Word Embeddings: Word Embeddings are a technique used to compress those massive 50,000-dimensional sparse arrays into small, dense, continuous coordinate spaces (usually 100 to 300 dimensions). Instead of a word being a single 1 in a sea of zeros, a word becomes a dense array of floating-point numbers: [-0.25, 0.52, 0.125, 0.2569, 0.241...].

Word2Vec: In 2013, there was a breakthrough architecture that proved how to calculate these dense arrays efficiently. By building a shallow neural network and forcing it to predict the surrounding context of a target word (**Skip-Gram model**), the network's internal weights naturally organized themselves into a geometric map of human meaning. Because Word2Vec forces words that appear in similar contexts to have similar vector coordinates, it unlocks the ability to do linear algebra on human language. For the first time, semantic relationships could be mathematically proven:

$$V(King) - V(Man) + V(Women) \approx V(Queen)$$

This Skip-Gram model is the main topic of this documentation.

The Skip-Gram Architecture:

To translate language into geometry, Word2Vec can be configured in two different ways: **Continuous Bag of Words (CBOW)**, which tries to guess a missing center word based on its surroundings, and **Skip-Gram**, which takes a single center word and tries to predict the surrounding context. For this build, we are implementing the Skip-Gram architecture because it performs exceptionally well with small datasets and creates highly accurate representations for rare words.

The algorithm of **Word2Vec** is really simple. In Word2Vec, our main goal is to create a meaningful vector space of words. For example, the vector for the word “house” and the vector for the word “home” should be the same or close to the same. Let's look at this sample image:

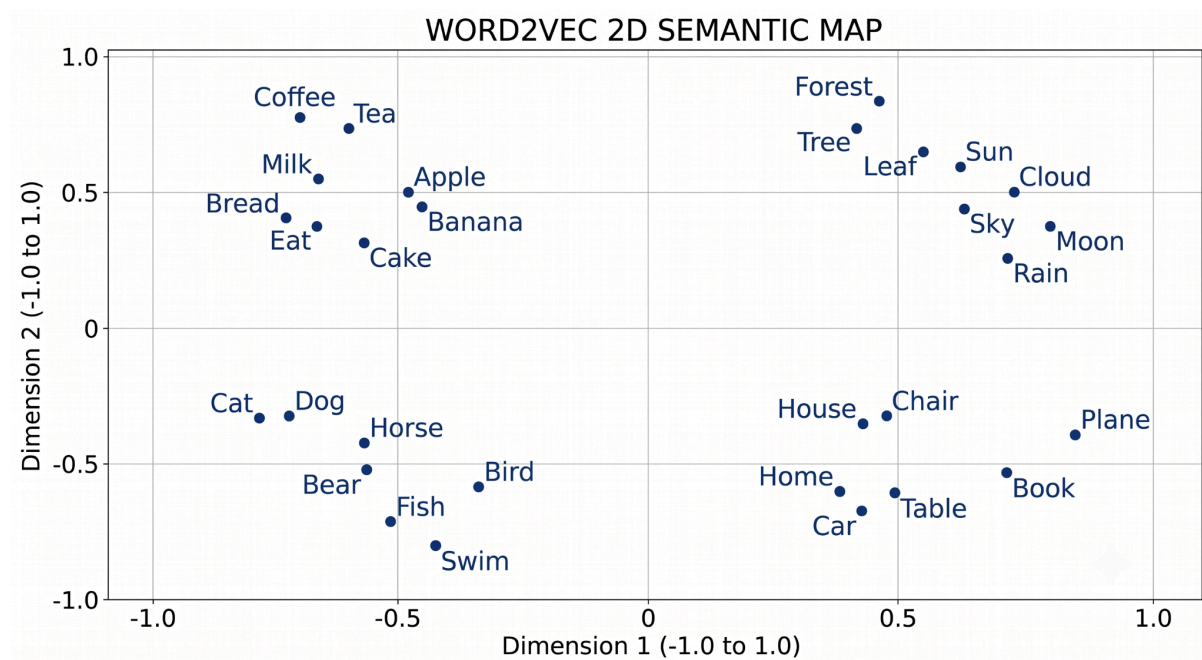


Fig 1.0 ~ 2D vectorspace showing word similarity.

Here, Drinks and food items, natural elements, and weather, animals, and related actions are close, because their vectors are similar (same direction and magnitude). Now this image

is just for understanding; the real Skip-Gram model has a vector of about 300 dimensions. For visualization, we used 2D vectors.

How do we generate this vector space? Well, we use a sliding window concept.

Sliding Context Window: We take each word, process its surrounding words, and then calculate the mathematical probability of those specific neighbors appearing together. Let's take an example:

"We bought a new home."

["We", "brought", "a", "new", "home"] (Indices: 0, 1, 2, 3, 4)

Let's assume we configure our hyperparameter for the window size to 2, meaning the algorithm looks at a maximum of two words to the left and two words to the right. So we traverse through the array, for "we", we look at "brought", "a". Then for "brought" we look at "we", "a", "new" and so on.

The model's (Skip-Gram) job is to learn that these words come together. Let's define our objective function,

For each position $t = 1, \dots, T$, predict context words within a window of fixed size m , given the center word w_t . Data likelihood:

$$L(\theta) = \prod_{t=1}^T \prod_{-m \leq j \leq m (j \neq 0)} p(w_{t+j} | w_t; \theta)$$

This gigantic equation is simply saying, "from $t = 1$ to T , look at m words left and right and calculate the probability of the central word with the given context word."

In our example, ["We", "brought", "a", "new", "home"]. Let's use "new" as our central word. For "new" index is 3. So $t = 3$. " m " will be -2 to 2 as our context window is 2 words long. So " j " will be -2, -1, 1, 2 (cannot be 0 as 0 is the center word). So it gives us the probability, $P(w_{3-2=1} | w_3)$ which is nothing but the $P(\text{brought} | \text{new})$, or we can say how likely "brought" and "new" come together. The same thing will happen for $j = -1, 1, 2$. Gives us the likelihood for "new" with "a" and "home". Since there are no words, $j=2$ will be ignored. The model's job is to maximize the probability.

Now, as we know in Deep learning, optimizers are designed to minimize the loss, not maximize, so we use a technique called **negative log likelihood**.

$$J(\theta) = -\frac{1}{T} \log(L(\theta)) = -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m (j \neq 0)} \log(P(w_{t+j} | w_t; \theta))$$

Now, **minimizing the objective function (J) = maximizing predictive accuracy**.

NOTE: In the mathematical notation $P(w_{t+j} | w_t; \theta)$, the symbol θ represents all learnable parameters in the model (weights and biases), and the **semicolon (;)** means "parameterized by." Read together, it simply instructs the algorithm to calculate the probability using the current network weights.

Now, how to calculate $P(w_{t+j} | w_t; \theta)$?

We will use two vectors per word W :

- v_W When W is a center word
- u_W When W is a context word

These word vectors are a subpart of the big vector of all parameters θ .

So, a center word c and a context window word o :

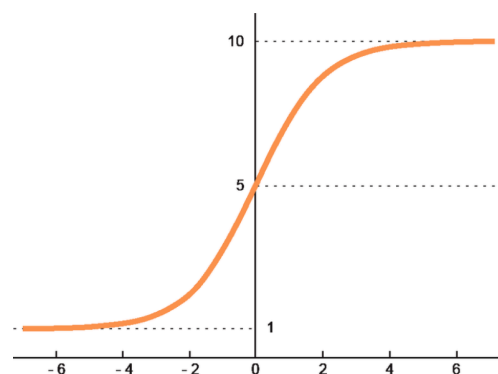
$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

Here, this dot product compares the similarity of o and c . The larger the dot product, the larger the probability. $\sum_{w \in V} \exp(u_w^T v_c)$ This is used to normalize over the entire vocabulary to give a probability distribution.

Now, it is an example of the **softmax** function. $R^n \rightarrow (0, 1)^n$

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)} = p_i$$

It maps the arbitrary values x_i to a probability distribution p_i .



So, in the Skip-Gram model, we feed a bunch of texts, and it will calculate each word from each sentence and try to guess the probability of the center word coming next to the context word. Of course, as both words are next to each other, the probability should be 1, but on a feed-forward network, it cannot generate 1, and so we need to calculate the loss and readjust those weights and biases so that it will generate 1.

Backpropagation on Skip-Gram model:

To find the minima, we have to partially differentiate the objective function J or the negative log likelihood function with respect to each central word.

$$\frac{\partial J}{\partial V} = - \frac{1}{T} \frac{\partial(\log(L(\theta)))}{\partial V}$$

You can solve this by yourself; it's just a basic calculus problem. I'll jump straight to the result, and it is wonderful. The result of this partial derivative is actually a beautiful equation in Machine Learning.

$$\frac{\partial J}{\partial V} = \sum_{x=1}^V P(x|c)u_x - u_o$$

In pure English, this translates to: **Gradient = (Expected Context Vector) - (Observed Context Vector)**

The model looks at its own Softmax probabilities, multiplies every single word vector in the dictionary by its respective probability, and adds them all up. This creates an "average" vector of what the model *thought* the context should look like. By subtracting the observed reality from the model's expectation, we get a directional gradient. It tells the center vector exactly which way to move in the multi-dimensional space to fix its mistake.

Similarly, the gradient for every context word (u_w) in the vocabulary is calculated as:

$$\frac{\partial J}{\partial u_w} = (P(w|c) - y_w)V_c$$

Where y_w is exactly 1.0 if w is the actual target context word, and 0.0 for every other word in the dictionary.

Now, we have enough knowledge to build our own Word2Vec from scratch in C. I have written the whole Code and explained it in comments, which are easy to understand.

Let's look at the code then.

```
#include <stdio.h>
#include <stdlib.h>
```

```

#include <string.h>
#include <math.h>
#include <time.h>

// --- HYPERPARAMETERS ---
#define VOCAB_SIZE 10000    // Total number of unique words in our dictionary
#define EMBEDDING_DIM 300   // The size of our continuous vector space (300
// coordinates per word)
#define CONTEXT_WINDOW 2    // How many words to look at on the left and right
#define LEARNING_RATE 0.01  // Step size for Gradient Descent

// Forward declaration so the compiler knows this function exists
double _probability_denominator(double *center_word, int dim);

// --- THE WEIGHT MATRICES (Theta) ---
// These two arrays hold the raw floating-point coordinates for our vocabulary.
double U[VOCAB_SIZE][EMBEDDING_DIM]; // Context Word Matrix (Target words)
double V[VOCAB_SIZE][EMBEDDING_DIM]; // Center Word Matrix (Anchor words)

// Initializes the matrices with random noise between -1.0 and 1.0
void init_embeddings() {
    for (int i = 0; i < VOCAB_SIZE; i++) {
        for (int j = 0; j < EMBEDDING_DIM; j++) {
            U[i][j] = ((double)rand() / RAND_MAX) * 2.0 - 1.0;
            V[i][j] = ((double)rand() / RAND_MAX) * 2.0 - 1.0;
        }
    }
}

// Computes the geometric similarity between two vectors.
// A higher positive result means the vectors point in the same direction.
double dot_product(double *vec1, double *vec2, int dim) {
    double result = 0.0;
    for (int i = 0; i < dim; i++) {
        result += vec1[i] * vec2[i];
    }
    return result;
}

// --- THE SOFTMAX FUNCTION (Forward Pass) ---
// Calculates P(Context | Center): The probability of the context word given the
// center word.
double probability(double *center_word, double *vec2, int dim) {
    // 1. Calculate raw similarity (numerator)
    double dot = dot_product(center_word, vec2, dim);
    double numerator = exp(dot);

    // 2. Normalize against the entire dictionary (denominator)
    double denominator = _probability_denominator(center_word, dim);

    // 3. Return a strict percentage between 0.0 and 1.0
    double prob = numerator / denominator;
}

```

```

        return prob;
    }

    // Calculates the global baseline by dotting the center word against EVERY word in
    // the dictionary
    double _probability_denominator(double *center_word, int dim) {
        double sum = 0.0;
        for (int i = 0; i < VOCAB_SIZE; i++) {
            double dot = dot_product(center_word, U[i], dim);
            sum += exp(dot);
        }
        return sum;
    }

    // --- LOSS CALCULATION ---
    // Measures how wrong the network's current predictions are across the entire text
    // sequence
    double negative_log_likelihood(int *corpus, int corpus_size) {
        double loss = 0.0;
        int valid_windows = 0;

        // Slide the context window across the text corpus
        for(int i = 0; i < corpus_size; i++){
            int center_idx = corpus[i];
            double sectional_sum = 0.0;

            for(int j = -CONTEXT_WINDOW; j <= CONTEXT_WINDOW; j++){
                if(j == 0) continue; // Skip the center word
                int context_pos = i + j;

                // Ensure the context word is within the text boundaries
                if(context_pos >= 0 && context_pos < corpus_size) {
                    int context_idx = corpus[context_pos];

                    // Get the model's predicted probability for this pair
                    double prob = probability(V[center_idx], U[context_idx],
EMBEDDING_DIM);

                    // Add the log of the probability (transforms multiplication to
addition)

                    sectional_sum += log(prob);
                    valid_windows++;
                }
            }
            loss += sectional_sum;
        }

        // Return the average NLL. We multiply by -1 to frame it as a minimization
        // problem.
        return - (loss / valid_windows);
    }

```

```

// --- BACKPROPAGATION ---
// Calculates gradients (Expected vs. Observed) and updates the vector coordinates
void backward_pass(int center_idx, int target_idx){

    // Stack-allocated array to accumulate the gradient for the center vector
    double grad_v[EMBEDDING_DIM] = {0.0};

    // Pre-compute the Softmax denominator once to save CPU cycles
    double denom = _probability_denominator(V[center_idx], EMBEDDING_DIM);

    // Loop through the entire vocabulary to calculate Expected vs. Observed errors
    for(int i = 0; i < VOCAB_SIZE; i++){

        // 1. EXPECTED: What probability did the model assign to this word?
        double prob = exp(dot_product(V[center_idx], U[i], EMBEDDING_DIM)) / denom;

        // 2. OBSERVED: Did this word actually appear in the text? (1.0 if Yes, 0.0
if No)
        double observed = (i == target_idx) ? 1.0 : 0.0;

        // 3. ERROR: The gradient direction
        double error = prob - observed;

        // 4. Update the Context Matrix (U) and accumulate the Center gradient (V)
        for(int j = 0; j < EMBEDDING_DIM; j++){
            // Accumulate: error * context_vector
            grad_v[j] += error * U[i][j];

            // Gradient Descent for Context: U_new = U_old - (lr * error * V_center)
            U[i][j] -= LEARNING_RATE * error * V[center_idx][j];
        }

        // 5. Update the Center Matrix (V) using the fully accumulated gradient
        for (int d = 0; d < EMBEDDING_DIM; d++) {
            V[center_idx][d] -= LEARNING_RATE * grad_v[d];
        }
    }
}

int main() {
    srand(time(NULL));

    // Initialize the matrices with random noise
    init_embeddings();

    // --- THE MOCK TOKENIZER ---
    // To humans, the sentence is: "the cat sat on the mat."
    // We map each unique word to a strict integer ID.
    // "the" = 0, "cat" = 1, "sat" = 2, "on" = 3, "mat" = 4

    int tokenized_corpus[] = {0, 1, 2, 3, 0, 4};
    int corpus_size = sizeof(tokenized_corpus) / sizeof(tokenized_corpus[0]);

```



```

printf("Starting Training...\n\n");

int epochs = 50;

// --- THE TRAINING LOOP ---
for (int epoch = 0; epoch < epochs; epoch++) {

    // Slide the window across the tokenized text array
    for (int i = 0; i < corpus_size; i++) {
        int center_idx = tokenized_corpus[i];

        for (int j = -CONTEXT_WINDOW; j <= CONTEXT_WINDOW; j++) {
            if (j == 0) continue; // Skip the anchor word

            int context_pos = i + j;

            // If the context word is within the bounds of the sentence
            if (context_pos >= 0 && context_pos < corpus_size) {
                int target_idx = tokenized_corpus[context_pos];

                // Run backpropagation to adjust the geometry of this specific
word pair
                backward_pass(center_idx, target_idx);
            }
        }

        // Print the loss every 10 epochs to verify Gradient Descent is working
        if (epoch % 10 == 0 || epoch == epochs - 1) {
            double current_loss = negative_log_likelihood(tokenized_corpus,
corpus_size);
            printf("Epoch %d | NLL Loss: %f\n", epoch, current_loss);
        }
    }

    printf("\nTraining Complete. Meaning has been vectorized.\n");
    return 0;
}

```

Now, if you run it, the output will be something like this:

```

Starting Training...
Epoch 0 | NLL Loss: 17.532775
Epoch 10 | NLL Loss: 1.590746
Epoch 20 | NLL Loss: 1.468074
Epoch 30 | NLL Loss: 1.453119
Epoch 40 | NLL Loss: 1.445786
Epoch 49 | NLL Loss: 1.441015

```

Training Complete. Meaning has been vectorized.

I used AI tools to add comments elaborately. I hope you find this useful. That's all for Word2Vec.

AUTHOR: Aritra Das